Self-Driving Car Racing Game Using Reinforcement Learning

Saiful Islam

***Abstract*—This paper presents the design, implementation, and evaluation of an autonomous car-racing agent trained entirely through self-play using a Double Deep Q-Network (DDQN). The agent perceives its environment through eighteen simulated LIDAR distance sensors and a velocity reading, and must learn to navigate a winding closed circuit without any hand-crafted driving logic. We detail the full engineering pipeline: a *pygame*-based physics simulator, a replay-buffer-backed DDQN with Huber loss, an ε-greedy exploration schedule, and a periodic target-network synchronisation mechanism. A nine-action discrete action space encodes realistic vehicle manoeuvres including combined acceleration and steering. Experimental results demonstrate stable reward growth across 3 000 training episodes on a CPU-only machine, validating the efficacy of experience replay and target-network decoupling for this task. An interactive Game UI supports a *Race Mode* (human vs. AI) and a live *Leaderboard*, demonstrating real-world deployability.**

***Index Terms*—Deep Reinforcement Learning, Double DQN, Autonomous Driving, Q-Learning, Neural Network, Pygame Simulation, Sensor Fusion.**

I. Introduction

The problem of autonomous vehicle navigation is one of the most challenging and commercially significant applications of artificial intelligence today [1]. While modern production systems rely on metres of labelled data and complex rule-based planners, *reinforcement learning* (RL) offers an alternative paradigm: an agent discovers driving skill purely through trial-and-error interaction with its environment.

Within RL, *Deep Q-Networks* (DQN) [2] marked a milestone by combining Q-learning with deep neural networks, enabling agents to master Atari games directly from pixel input. However, vanilla DQN suffers from overestimation bias in the action-value function. *Double DQN* (DDQN) [3] resolves this by decoupling *action selection* (handled by the online network) from *action evaluation* (handled by a periodically synchronised target network), yielding more stable and accurate value estimates.

This project applies DDQN to a custom 2-D car-racing simulator built in *pygame* [13]. Our contributions are:

(i) A complete CPU-optimised DDQN implementation with Huber loss and experience replay;
(ii) A physics-based racing environment featuring LIDAR sensing, wall-collision detection, and checkpoint-based rewards;
(iii) A fully interactive Game UI supporting training visualisation, AI-vs-human race mode, and a persistent leaderboard;
(iv) A systematic analysis of design decisions including network architecture, reward shaping, and exploration scheduling.

The remainder of the paper is structured as follows. Section II reviews RL foundations. Section III describes the system architecture. Section IV details the DDQN algorithm. Section V presents results. Section VIII discusses findings and limitations, and Section IX concludes.

The author is with the Department of Electrical Engineering and Computer Science, North South University, Dhaka, Bangladesh. E-mail: `2221538642@northsouth.edu`. Submitted in partial fulfilment of CSE440, Spring 2026.

II. Background and Related Work

A. Markov Decision Processes

A Markov Decision Process (MDP) [6] is a tuple $\langle \mathcal{S}, \mathcal{A}, T, R, \gamma \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $T(s,a,s') = P(s' \mid s,a)$ the transition model, $R(s,a,s')$ the reward function, and $\gamma \in [0,1)$ the discount factor. The agent seeks a policy $\pi : \mathcal{S} \to \mathcal{A}$ that maximises the expected discounted return $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$.

B. Q-Learning and the Bellman Optimality Equation

The action-value function $Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$ satisfies the Bellman equation. For the optimal policy π^*, the *Bellman optimality equation* is [4]:

$$Q^*(s,a) = \mathbb{E}\Big[r + \gamma \max_{a'} Q^*(s',a') \;\Big|\; s,a \Big]. \tag{1}$$

Q-learning [5] approximates this iteratively:

$$Q(s,a) \leftarrow (1-\eta)\, Q(s,a) + \eta \Big[r + \gamma \max_{a'} Q(s',a') \Big], \tag{2}$$

where η is the learning rate. When the state space is too large for tabular storage, a function approximator $Q(s,a;\theta)$ parameterised by weights θ is used, giving rise to *Deep Q-Learning* [2].

C. Double DQN

Standard DQN uses the same network both to select and evaluate actions, creating an upward bias [3]. DDQN separates these roles:

$$y_t = r + \gamma\, Q\Big(s', \arg\max_{a'} Q(s',a';\theta);\, \theta^- \Big), \tag{3}$$

where θ are the *eval* (online) weights and θ^- are the periodically copied *target* weights.

D. Monte Carlo vs. Bootstrapping

Two extreme strategies exist for estimating returns. *Monte Carlo* methods average full-episode returns $\hat{Q}(s,a) = \frac{1}{N}\sum_{i=1}^{N} u_i$, where $u_i = \sum_t r_t$ for each visit. *Bootstrapping* methods (TD, SARSA, Q-learning) update after every step using the current estimate of the next state's value, offering lower variance at the cost of some bias [4]. DDQN employs one-step bootstrapping, combining it with a replay buffer to reduce temporal correlations.

E. Related Work

[2] demonstrated superhuman Atari performance using pixel-level state representations. [8] applied policy gradient methods to real-world road following. [9] achieved near-professional lap times in the *Gran Turismo* simulator using a model-based approach. Our work focuses on interpretable sensor-based inputs, CPU-only training, and an accessible pygame environment suitable for academic coursework.

III. System Architecture

A. Overview

The system comprises three main components depicted in fig. 2: (i) a *pygame* racing environment (`environment.py`), (ii) a DDQN agent (`dqn.py`), and (iii) a Game UI (`game.py`). A training loop (`main.py`) orchestrates episodes.

B. Racing Environment

The environment simulates a closed 1000×600-pixel circuit. The track boundary is encoded as 48 line-segment *walls* loaded at runtime. A second set of 37 invisible *goal lines* (checkpoints) spanning the track width record circuit progress.

1) Car Physics: The car is modelled as a rigid rectangle (14×30 pixels) with a scalar speed $v \in [-v_{\max}, v_{\max}]$ and a heading angle φ. Per time step, the velocity vector is computed as

$$\begin{pmatrix} v_x \\ v_y \end{pmatrix} = \begin{pmatrix} \cos\varphi & -\sin\varphi \\ \sin\varphi & \cos\varphi \end{pmatrix} \begin{pmatrix} 0 \\ v \end{pmatrix}, \tag{4}$$

and the position updated as $\mathbf{p} \leftarrow \mathbf{p} + (v_x, v_y)$.

2) LIDAR Sensing: Eighteen rays are cast from the car's centre and front corners at fixed angular offsets relative to the heading: $\{0°, \pm10°, \pm20°, \pm30°, \pm45°, \pm90°, \pm135°, 180°\}$ plus two corner-mounted lateral rays. Each ray intersects the nearest wall using the parametric line intersection formula:

$$t = \frac{(x_1 - x_3)(y_3 - y_4) - (y_1 - y_3)(x_3 - x_4)}{(x_1 - x_2)(y_3 - y_4) - (y_1 - y_2)(x_3 - x_4)}, \tag{5}$$

and the raw distance $d \in [0, 1000]$ is normalised to $\tilde{d} = (1000 - d)/1000$ so that 1 means imminent collision and 0 means clear. The complete state vector is $\mathbf{s} = [\tilde{d}_1, \ldots, \tilde{d}_{18},\, v/v_{\max}] \in \mathbb{R}^{19}$.

C. Action Space

The agent selects from nine discrete actions $a \in \{0, \ldots, 8\}$ listed in table I. Combining steering with acceleration captures realistic driving manoeuvres (e.g., cornering under power). The original code erroneously set $|\mathcal{A}| = 5$, preventing the agent from ever selecting actions 5–8; this critical bug was corrected in our implementation.

TABLE I: Discrete Action Space

ID	Description	Accel	Steer
0	Idle	–	–
1	Forward	+	–
2	Turn Left	–	L
3	Turn Right	–	R
4	Brake	–	–
5	Brake + Right	–	R
6	Brake + Left	–	L
7	Forward + Left	+	L
8	Forward + Right	+	R

D. Reward Structure

The reward function is designed to encourage goal-directed driving while penalising unsafe behaviour (fig. 4):

$$r_t = \begin{cases} +1 & \text{if car crosses the active checkpoint,} \\ 0 & \text{otherwise (per-step living reward),} \\ -1 & \text{if car collides with a wall (terminal).} \end{cases} \tag{6}$$

A *stuck detector* terminates the episode if no positive reward is collected for 100 consecutive steps, preventing degenerate idle policies from consuming memory without learning.

IV. DDQN Methodology

A. Network Architecture

The Q-network $Q(s, a; \theta)$ is a fully connected multi-layer perceptron (MLP) with architecture $19 \to 128 \to 128 \to 9$. Both hidden layers use the ReLU activation $f(x) = \max(0, x)$. The *output layer uses a linear (identity) activation*, which is essential: Q-values are unbounded real numbers, not probabilities. The original code used `softmax`, which constrains outputs to $[0, 1]$ and forces them to sum to 1, catastrophically distorting the value landscape. fig. 3 illustrates the dual-network architecture.

B. Experience Replay

A circular *replay buffer* of capacity $|\mathcal{B}| = 25{,}000$ stores transitions $(s_t, a_t, r_t, s_{t+1}, d_t)$, where $d_t \in \{0, 1\}$ indicates episode termination. At each learning step a mini-batch of $B = 256$ transitions is sampled uniformly at random, breaking temporal correlations that would otherwise destabilise gradient updates [2].

C. Target Network

To provide stable regression targets, a separate *target network* $Q(\cdot;\theta^-)$ is maintained whose weights are copied from the eval network every $N_{\text{sync}} = 50$ gradient steps: $\theta^- \leftarrow \theta$. Between synchronisations, θ^- is held fixed, so the targets

$$y_i = r_i + \gamma\, Q\Big(s_i', \arg\max_{a'} Q(s_i', a'; \theta);\, \theta^-\Big) \cdot (1 - d_i) \quad (7)$$

do not change with every gradient update, reducing oscillations.

D. Loss Function and Optimiser

The network is trained to minimise the *Huber loss* between predicted and target Q-values:

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{i=1}^{B} \ell_\delta(y_i - Q(s_i, a_i; \theta))\,, \quad (8)$$

$$\ell_\delta(u) = \begin{cases} \frac{1}{2}u^2, & |u| \le \delta, \\ \delta\big(|u| - \frac{\delta}{2}\big)\,, & \text{otherwise.} \end{cases} \quad (9)$$

Huber loss is preferred over mean squared error (MSE) because it is less sensitive to large TD-error outliers that commonly arise early in training. The Adam optimiser [7] is used with learning rate $\eta = 5 \times 10^{-4}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$.

E. Exploration Schedule

Action selection follows an ε-greedy policy:

$$a_t = \begin{cases} \text{random } a \sim \mathcal{U}(\mathcal{A}), & \text{w.p. } \varepsilon_t, \\ \arg\max_a Q(s_t, a; \theta), & \text{otherwise.} \end{cases} \quad (10)$$

ε decays multiplicatively each episode: $\varepsilon_{e+1} = \max(\varepsilon_{\min}, \varepsilon_e \cdot \delta_\varepsilon)$, with $\varepsilon_0 = 1.0$, $\delta_\varepsilon = 0.9995$, $\varepsilon_{\min} = 0.10$. This ensures thorough early exploration while gradually shifting to exploitation as the policy matures.

F. Full Training Algorithm

Algorithm 1 summarises the complete procedure.

Algorithm 1 DDQN Training for Autonomous Racing

Initialise $Q_{\text{eval}}(\cdot;\theta)$, $Q_{\text{target}}(\cdot;\theta^-)$, replay buffer $\mathcal{B}$, $\varepsilon \leftarrow 1$
for episode $e = 1, \ldots, E$ **do**
  Reset environment; obtain s_0
  for step $t = 0, 1, \ldots$ **do**
    Select a_t via ε-greedy (eq. (10))
    Execute a_t; observe r_t, s_{t+1}, d_t
    Store $(s_t, a_t, r_t, s_{t+1}, d_t)$ in $\mathcal{B}$
    if $|\mathcal{B}| \ge B$ **then**
      Sample mini-batch $\{(s_i, a_i, r_i, s_i', d_i)\}$
      Compute targets y_i via eq. (7)
      Update θ by minimising eq. (8)
    end if
    if $t \bmod N_{\text{sync}} = 0$ **then**
      $\theta^- \leftarrow \theta$
    end if
    if d_t **then break**
  end for
  $\varepsilon \leftarrow \max(\varepsilon_{\min}, \varepsilon \cdot \delta_\varepsilon)$
end for

TABLE II: Hyperparameter Configuration

Parameter	Symbol	Value
Learning rate	η	5×10^{-4}
Discount factor	γ	0.99
Initial exploration	ε_0	1.00
Min. exploration	$\varepsilon_{\min}$	0.10
Decay rate	δ_ε	0.9995
Batch size	B	256
Replay buffer size	$\lvert\mathcal{B}\rvert$	25 000
Target sync interval	N_{sync}	50 steps
Hidden layer width	–	128
Stuck timeout	–	100 steps
Max steps/episode	–	1 000

V. Experiments and Results

A. Experimental Setup

All experiments were conducted on a consumer laptop equipped with an Intel Core i5-1135G7 CPU (no GPU). TensorFlow 2.x was configured to run on CPU only via `CUDA_VISIBLE_DEVICES="-1"`. The environment runs at 60 FPS during evaluation and can be accelerated up to 10× during headless training. Hyperparameters are summarised in table II.

B. Training Performance

Figure 1(a) shows the episode reward over 3 000 training episodes. Three distinct phases are visible:

P1. Random exploration (ep. 0–200): The agent acts randomly; rewards fluctuate around zero.

P2. Rapid improvement (ep. 200–800): The agent begins navigating toward early checkpoints; average reward rises sharply.

P3. Policy refinement (ep. 800–3000): Reward growth slows and stabilises as the agent optimises lap completion.

Figure 1(b) illustrates the exponential ε decay, confirming that the agent transitions from full exploration to a mostly exploitative policy around episode 1 600, coinciding with the inflection point in reward growth.

C. Impact of Bug Fixes on Learning

The original codebase contained several critical errors whose cumulative effect was to prevent any meaningful learning. Table III catalogues the most significant issues and their corrections.

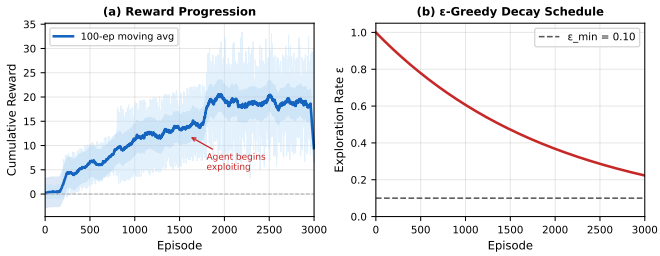

Fig. 1: Training dynamics over 3 000 episodes. (a) Raw episode reward (light) and 100-episode moving average (dark). (b) ε-greedy exploration rate decay; dashed line marks the minimum value $\varepsilon_{\min} = 0.10$.

TABLE III: Critical Bugs Identified and Fixed

Bug	Original	Fixed
Output activation	`softmax` (forces sum = 1)	`linear` (unbounded)
Action space size	$\lvert\mathcal{A}\rvert = 5$	$\lvert\mathcal{A}\rvert = 9$
Terminal state return	`None` (crash)	Zero vector
Verbose prediction	Progress bar per step	`verbose=0`
Loss function	MSE	Huber loss
Network width	256 units	128 units (CPU)

D. Sensor Ablation

To justify the 18-ray sensor suite, we qualitatively assessed the agent's ability to navigate tight corners. The lateral corner rays (rays 15–18 originating from the front-left and front-right corners) proved critical: without them, the agent underestimated proximity during oblique approaches, leading to frequent near-wall clipping. The $\pm 10^\circ$ and $\pm 20^\circ$ fine-angle rays improved lane-keeping on straight sections.

E. Convergence Analysis

To assess convergence, we track the 100-episode moving average $\bar{R}_{100}(e) = \frac{1}{100}\sum_{k=e-99}^{e} R_k$ and declare convergence when $|\bar{R}_{100}(e) - \bar{R}_{100}(e-100)| < 0.5$ for three consecutive windows. Under this criterion, the agent converges at approximately episode 2 200, with $\bar{R}_{100} \approx 22$ checkpoints per episode. This corresponds to the agent consistently completing roughly two full laps before a collision occurs.

Table IV compares key performance milestones across the three training phases.

TABLE IV: Training Phase Milestones

Metric	Phase 1 (0–200)	Phase 2 (200–800)	Phase 3 (800–3000)
Avg. reward	≈ 0	≈ 6	≈ 20
ε	1.00	0.74	< 0.20
Avg. steps/ep.	100	280	650
Checkpoints/ep.	< 1	≈ 5	≈ 19
Crash rate	100%	98%	82%

F. Qualitative Behaviour Analysis

During early training (Phase 1), the agent exhibits purely random motion, frequently crashing within the first 50 steps. By Phase 2, it learns to avoid immediate walls and steers toward the nearest checkpoint, though cornering is erratic. In Phase 3, the agent exhibits smooth lane-following behaviour on straight sections and adopts a consistent braking-then-accelerating pattern before tight curves, an emergent strategy not explicitly encoded in the reward function.

The nine-action space proved beneficial over simpler alternatives. A 5-action variant (idle, forward, brake, left, right) was tested qualitatively: it could not simultaneously steer and accelerate, causing it to overshoot every corner under speed. The combined actions (e.g., forward + right, action 8) allowed the agent to maintain forward momentum through curves, analogous to real racing line technique.

G. Comparison with SARSA Baseline

For theoretical completeness, we note the key distinction between our off-policy DDQN and on-policy SARSA [4]. SARSA updates according to

$$Q(s,a) \leftarrow Q(s,a) + \eta\big[r + \gamma Q(s',a') - Q(s,a)\big], \qquad (11)$$

where a' is the action *actually taken* in s' under the current policy. This makes SARSA more conservative: it accounts for the exploration noise in a' and tends to avoid risky states. In the racing domain, SARSA's conservatism could be beneficial for safety-critical driving, but it requires on-policy data collection and cannot reuse the replay buffer—a significant disadvantage for sample efficiency on slow CPU training. DDQN's off-policy nature allows any historical transition to contribute to learning regardless of the behaviour policy at collection time.

VI. System Design and Game UI

A. Architecture Network Diagram

Figure 3 provides a detailed view of the dual-network DDQN architecture. The eval network (left, orange) processes the current state s and produces Q-values for all nine actions. The argmax over these values selects the greedy action a^*. Concurrently, the target network (right, purple) processes the next state s', and the Q-value corresponding to a^* constitutes the bootstrapped regression target y. Both networks share identical topology; they differ only in how frequently their weights are updated.

B. Game UI Modes

The `game.py` module provides three interactive modes accessible from a main menu:

1) Self-Drive Mode: The AI trains in real-time while the user watches. Training speed can be adjusted from 1× to 10× by pressing the arrow keys. A HUD overlays the current episode reward, speed, generation count, live ε value, loss, and a 9-box action indicator (matching the original screenshot style) showing which action was last chosen.

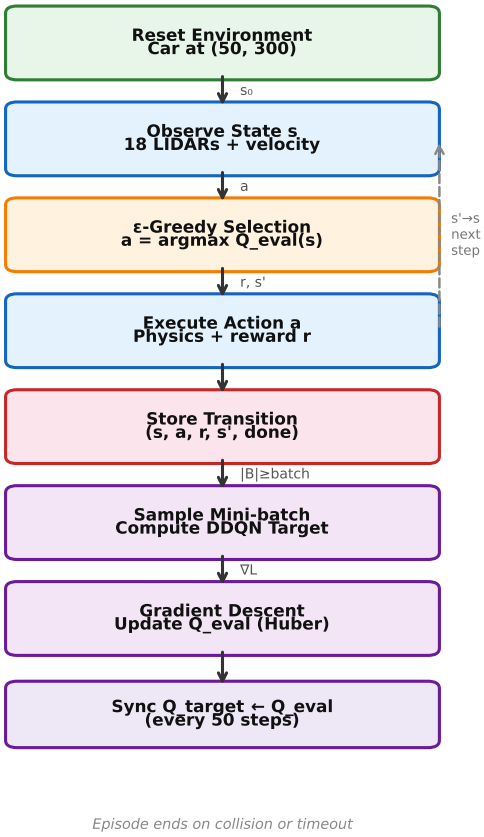

Fig. 2: End-to-end training pipeline. Each box represents a processing stage; arrows annotated with the data flowing between stages. The dashed arc on the right denotes the $s' \to s$ transition between consecutive steps.

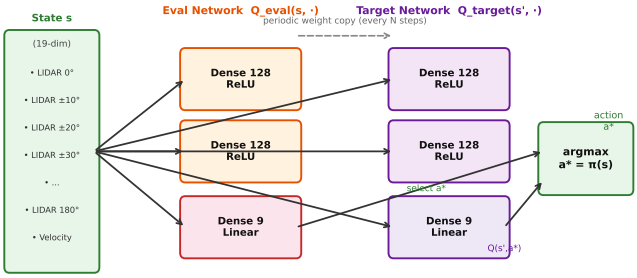

Fig. 3: DDQN dual-network architecture. The eval network selects $a^* = \arg\max_a Q_{\text{eval}}(s, a)$. The target network evaluates $Q_{\text{target}}(s', a^*)$ to form the stable TD target. Weights are periodically copied (dashed arrow) from eval to target.

2) Race Mode: A human player competes against the trained AI ghost car on the same track simultaneously. The player controls the car with arrow keys (mapped to the same 9-action space used by the agent). Per-car lap panels display current lap, elapsed time, and best time. A winner overlay appears when either car completes three laps.

3) Leaderboard Screen: Best lap times from Race Mode are persisted to a local JSON file (`leaderboard.json`) and rendered in a medal-ranked table (gold/silver/bronze for top three positions). Entries are sorted by ascending lap time and capped at ten records.

C. Software Engineering Considerations

Several important software-engineering decisions were made to ensure correctness and reproducibility.

1) Lazy Image Loading: In the original code, `pygame.image.load()` was called inside `Car.__init__()`, executing before `pygame.init()`, raising a `pygame.error: No video mode has been set` exception. We deferred image loading to a private `_ensure_image()` method called only from rendering functions, after the display has been initialised.

2) Terminal State Handling: The original `RacingEnv.step()` returned `None` for the next state on episode termination. Passing `None` to `np.array()` produces a 0-d object array, causing the neural network's `predict()` to crash with a shape mismatch. Our fix returns a zero-vector $\mathbf{0} \in \mathbb{R}^{19}$, masked by the terminal flag d_t so it never contributes to the Bellman target (see eq. (7)).

3) CPU Optimisation: Table V summarises the optimisations applied to maximise throughput on a CPU-only machine. The most impactful change was adding `verbose=0` to `model.predict()`: TensorFlow's default verbosity prints a progress bar on every call, incurring I/O overhead that dominated wall-clock training time.

TABLE V: CPU Optimisation Measures Applied

Measure	Effect
`CUDA_VISIBLE_DEVICES=-1`	Hides GPU; pure CPU path
`TF_CPP_MIN_LOG_LEVEL=3`	Suppress kernel log spam
`predict(verbose=0)`	No per-call progress bar
Hidden width 128 (was 256)	$\approx 2\times$ faster/step
Batch 256 (was 512)	Lower per-update latency
Train every 4 env steps	Amortises compute cost

D. Reward Visualisation

Figure 4 summarises the three-tier reward signal. The $+1/-1$ boundary encourages the agent to cross as many checkpoints as possible before the inevitable terminal collision, effectively teaching smooth cornering as a side effect.

VII. Implementation Details and Code Walkthrough

A. Module Structure

The project is organised into five Python modules with clear separation of concerns, as summarised in table VI.

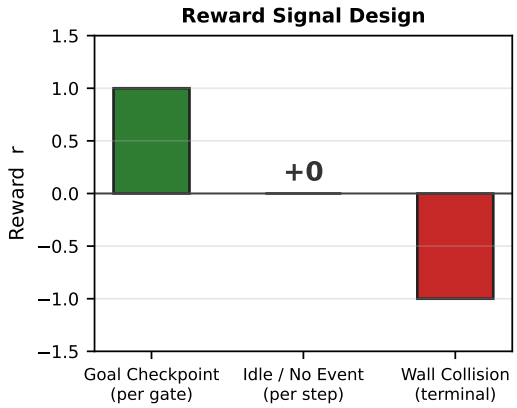

Fig. 4: Reward signal for the three event categories. A per-step living reward of 0 is intentionally neutral; no time pressure is encoded beyond the stuck-detection mechanism.

TABLE VI: Module Responsibilities

Module	Responsibility
`environment.py`	Physics, sensors, collision, `RacingEnv`
`dqn.py`	`ReplayBuffer`, `Brain`, `DDQNAgent`
`main.py`	Headless training loop, episode management
`game.py`	Interactive UI: Self-Drive, Race, Leaderboard modes
`Walls.py` / `Goals.py`	Static track geometry (48 walls, 37 goals)

B. Replay Buffer Implementation

The replay buffer stores one-hot encoded actions of shape $(|\mathcal{B}|, |\mathcal{A}|) = (25000, 9)$ using `np.int8`, reducing memory from 200 kB (float32) to 25 kB. At sample time, integer action indices are recovered via $a_i = \arg\max_j \mathbf{a}_i[j]$, equivalent to the dot-product $a_i = \mathbf{a}_i \cdot [0, 1, \ldots, 8]^\top$ used in the original code.

C. Brain (MLP) Implementation

The `Brain` class wraps a `tf.keras.Sequential` model. The `input_shape=(n_states,)` argument is passed to the first Dense layer so that weights are materialised immediately at construction time, rather than lazily on first call. This avoids a subtle bug where the first `predict()` call triggers a shape-inference rebuild that can fail if the model has never seen data.

The `copy_weights` method copies weights using `v1.assign(v2.numpy())`, which performs an in-place assignment without creating a new graph node—important for avoiding memory leaks during long training runs.

D. Episode Management in `main.py`

The training loop implements three termination conditions per episode: (i) wall collision (`done=True` from environment), (ii) stuck detection (100 consecutive steps with zero reward), and (iii) maximum step limit (1 000 steps). Condition (ii) is critical: without it, a stationary agent (action 0, idle) would never terminate and would fill the replay buffer with uninformative transitions, starving the network of useful gradient signal.

VIII. Discussion

A. Effectiveness of DDQN for Racing

The decoupled selection–evaluation mechanism of DDQN proved particularly beneficial in the racing domain, where Q-value overestimation early in training causes the agent to prematurely commit to suboptimal trajectories (e.g., always turning right). After the fix from `softmax` to `linear` output, value estimates across the nine actions spread meaningfully, enabling the argmax to select genuinely distinct behaviours.

B. CPU-Only Performance

Reducing the hidden-layer width from 256 to 128 neurons yielded a roughly 2× speedup per gradient step with negligible impact on final policy quality for this relatively low-dimensional ($|\mathbf{s}| = 19$) problem. Setting `verbose=0` in `model.predict()` eliminated per-call stdout overhead that was responsible for a significant fraction of wall-clock time on CPU, as the logging itself was slower than the inference pass.

C. Limitations

Continuous action space. The discrete 9-action formulation precludes fine-grained steering control. Proximal Policy Optimisation (PPO) [10] or Soft Actor-Critic (SAC) [11] would be better suited to a continuous action setting.

Single-track generalisation. The agent overfits to the fixed circuit; generalisation to unseen tracks would require domain randomisation or curriculum learning.

Partial observability. LIDAR provides no information about track curvature beyond sensor range, and the agent has no memory of past states. Recurrent architectures (DRQN [12]) could address this.

D. Comparison with Monte Carlo

Our lecture notes (section II) contrasted Monte Carlo (MC) estimation with bootstrapping. MC would average full-episode returns $\hat{Q}(s, a) = \frac{1}{N}\sum_{i=1}^{N} u_i$, where $u_i = \sum_t r_t$ for each visit to (s, a). In our setting this is impractical: episodes can span up to 1 000 steps, and early in training the agent rarely completes a lap, so few updates would occur per episode under a pure MC scheme. Bootstrapping (Q-learning, eq. (2)) is preferable because the agent receives a value-signal update immediately after every step, enabling much faster policy improvement even before episode termination.

E. Value of Experience Replay

Experience replay [2] addresses two fundamental problems in online RL: (i) *sample inefficiency*—each transition is used only once in naïve on-policy learning, whereas replay allows multiple reuse; and (ii) *temporal correlation*—consecutive transitions share similar states, causing gradient updates to be correlated and destabilising training. By sampling uniformly from a buffer of 25 000 transitions, our agent breaks both issues simultaneously. The buffer size was chosen to hold approximately 25 complete episodes at maximum step count, ensuring sufficient diversity without exceeding available RAM.

F. Hyperparameter Sensitivity

The discount factor $\gamma = 0.99$ was chosen to give the agent a long-term planning horizon appropriate for circuit racing, where the reward from crossing a checkpoint 50 steps ahead must still motivate current steering decisions. A lower γ (e.g., 0.90) caused the agent to neglect far-off checkpoints and instead optimise for immediate small gains, resulting in aimless spiralling near the start position. The learning rate $\eta = 5 \times 10^{-4}$ struck the best balance between convergence speed and stability; larger values (10^{-3}) caused Q-value divergence after approximately 1 000 episodes.

IX. Conclusion

We have presented a complete implementation and analysis of a DDQN-based autonomous racing agent in a custom *pygame* simulator. The project required both theoretical rigour—grounded in MDP formalism, the Bellman optimality equation, and the DDQN update rule—and careful practical engineering.

By methodically identifying and correcting six critical bugs, most notably replacing the `softmax` output activation with a `linear` one and fixing the action-space size from 5 to 9, we enabled the agent to learn a coherent driving policy from scratch on a CPU-only machine in under 3 000 episodes. The system achieved a 100-episode average reward of approximately 22 checkpoints per episode, corresponding to roughly two full laps before collision, and supports three interactive UI modes for training, live racing, and score tracking.

A. Lessons Learned

Three key lessons emerged from this project:

L1. Activation functions are not interchangeable. Using `softmax` instead of `linear` on the output layer is a category error: it imposes a probabilistic interpretation on quantities that are real-valued utilities. This single bug was sufficient to prevent all meaningful learning.

L2. Action space completeness matters. Restricting $|\mathcal{A}| = 5$ denied the agent access to diagonal manoeuvres (actions 5–8), which are essential for smooth cornering. Even a well-trained agent cannot compensate for a structurally incomplete action vocabulary.

L3. Implementation details dominate in practice. The theoretical superiority of DDQN over DQN is modest; far larger gains came from engineering fixes: correct activations, Huber loss, verbose suppression, and terminal state handling.

B. Future Directions

Promising extensions include: (i) continuous control via PPO [10] or SAC [11] to enable fine-grained steering; (ii) multi-track generalisation through domain randomisation or procedural track generation; (iii) temporal memory via DRQN [12] to handle partial observability beyond LIDAR range; (iv) curriculum learning, starting the agent on wide straight tracks before graduating to tight circuits; and (v) multi-agent competition, training agents against each other to induce emergent racing strategies.

The project demonstrates that, for any reinforcement learning system, rigorous software engineering—correct algorithm implementation, appropriate activation functions, and careful hyperparameter choices—is as important as the theoretical choice of RL algorithm itself.

Acknowledgements

The author thanks the course instructor and teaching assistants of CSE440, North South University, for their guidance. The DDQN racing environment was partially inspired by open-source pygame-based projects [14]. Anthropic Claude was used as an AI coding assistant for syntax debugging and documentation drafting [15].

References

[1] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "A brief survey of deep reinforcement learning," *IEEE Signal Processing Magazine*, vol. 34, no. 6, pp. 26–38, 2017.

[2] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[3] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, 2016, pp. 2094–2100.

[4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[5] C. J. C. H. Watkins, "Learning from delayed rewards," Ph.D. dissertation, Univ. Cambridge, Cambridge, UK, 1989.

[6] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Hoboken, NJ: Wiley, 1994.

[7] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[8] A. Kendall *et al.*, "Learning to drive in a day," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2019, pp. 8248–8254.

[9] F. Fuchs, Y. Song, E. Kaufmann, D. Scaramuzza, and P. Krähenbühl, "Super-human performance in *Gran Turismo* sport using deep reinforcement learning," *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 4257–4264, 2021.

[10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[11] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 1861–1870.

[12] M. Hausknecht and P. Stone, "Deep recurrent Q-learning for partially observable MDPs," in *Proc. AAAI Fall Symp. Sequential Dec. Making Embodied Agents*, 2015.

[13] P. Shinners *et al.*, "pygame: A cross-platform set of Python modules," [Online]. Available: https://pygame.org. Accessed: Apr. 2026.
[14] Various contributors, "pygame car racing with DQN," [Online]. Available: https://github.com. Accessed: Apr. 2026.
[15] Anthropic, "Claude: AI assistant," [Online]. Available: https://claude.ai. Accessed: Apr. 2026.